LaTeX News

Issue 39, June 2024 (LaTeX release 2024-06-01)

Contents

Introduction 1

News from the "LaTeX Tagged PDF" project 1

Enhancements to the new mark mechanism 2

Providing xtemplate in the format 2

New or improved commands 3
- doc: Provide `\ProvideDocElement` . . . 3
- doc: Better support for upquote . . . 3
- ifthen: Allow active characters in comparisons . 3
- New conditionals: `\IfClassAtLeastT` and friends . . . 3

Code improvements 3
- Load packages only at the top level . . . 3
- Keep track of lost glyphs . . . 3
- Improve fontenc error message . . . 4
- Warn if counter names are problematic . . . 4
- Extended information in `\listfiles` . . . 4
- Optimize creation of simple document commands . . . 4
- Handling of end-of-lines in `+v` arguments of `\NewDocumentCommand` and friends . . . 4
- Declaring appropriate sub-encodings for `TS1` symbol fonts . . . 4
- Behavior when loading textcomp without options 5
- Rollback improvements . . . 5

Documentation improvements 5
- Further updates to the guides . . . 5

Bug fixes 5
- Fix inconsistent expansion of the package option list . . . 5
- Fix logic for first mark (page region) . . . 6
- Struts at the end of footnotes or `p` columns . . 6
- Fix a "missing `\item`" rollback error . . . 6

Changes to packages in the amsmath category 6
- amsmath: Correct equation tag placement . . . 6

Changes to packages in the tools category 6
- array, longtable, tabularx: Support tagging . . . 6
- array: No `\unskip` in math cells . . . 6
- verbatim: `\verb` showed visible spaces . . . 6
- verbatim: Support tabs in `\verbatiminput*` . . 6
- multicol: `\columnbreak` interferes with mark mechanism . . . 6
- showkeys: Allow `\newline` in amsthm to work . 6
- xr: Support links and properties . . . 6

Changes to files in the cyrillic category 7
- Correct definition of `\k` . . . 7

Introduction

The LaTeX Project team remains strongly focused on producing automatically tagged PDF output for accessibility and reuse. At the beginning of 2024 the ISO PDF/UA-2 and the WTPDF (well-tagged PDF) standards were released and we are glad to be able to report that it is now possible to use LaTeX to automatically produce documents that conform to these new standards.[1] A sample collection of such documents ranging from classical texts, such as the Bible, to recent technical papers submitted to arXiv.org can be found at https://github.com/latex3/tagging-project/discussions/72.

In February Ulrike and Frank presented the current project status during the 5th International Workshop on "Digitization and E-Inclusion in Mathematics and Science 2024" (DEIMS 2024) at Nihon University, Tokyo, Japan; see [8].

News from the "LaTeX Tagged PDF" project

In the previous LaTeX News [7] we announced some prototype support for tagged tabulars. Some of the necessary code has now been moved from `latex-lab` to the corresponding packages (using sockets and plugs) and to the LaTeX kernel (for those parts that are also necessary for other aspects of tagging).

The kernel code specific to tagging is implemented in the file `lttagging.dtx`. For now it contains `\UseTaggingSocket`, a special invocation command for sockets that are specific to tagging. This enables us to also provide `\SuspendTagging` and `\ResumeTagging`, i.e., a very efficient way to temporarily disable the whole tagging process. This is, for example, necessary

[1] At the present time we are still in a trial/prototype phase in which only a limited set of document classes and packages are supported. Over the next releases we expect to gradually lift these restrictions and eventually provide the full functionality as part of the core distribution, rather than through `latex-lab` modules.

LaTeX News, and the LaTeX software, are brought to you by the LaTeX Project Team; Copyright 2024, license LPPL.

if some code is doing trial typesetting. In that case the trials should not generate tagging structures—only the finally-chosen version should. Thus, tabularx, for example, stops the tagging while doing its trials to figure out the correct column widths to use, and then re-enables tagging when the table is finally typeset.

Over time, `lttagging.dtx` will hold more general tagging code as appropriate. For now it is only documented as part of `source2e.pdf` but long term we will provide a separate guide for tagging, which will then also include the information currently found in various other places, e.g., `tagpdf.pdf`.

We also added support for a few missing commands described in Leslie Lamport's *LaTeX Manual* [1]: If `phase-III` is used the `\marginpar` command will be properly tagged (depending on the PDF version) as an `Aside` or a `Note` structure. In the standard classes `\maketitle` will be tagged if the additional testphase module `title` is used.

The `math` module has been extended and now includes options to attach MathML files to the structures. First tests with a PDF reader and screen reader that support associated files look very promising. Examples of PDF files tagged with the new method can be found at https://github.com/latex3/tagging-project/discussions/72.

At last various small bugs and problems reported at https://github.com/latex3/tagging-project have been fixed. Such feedback is very valuable, so we hope to see you there and thank you for any contribution, whether it is an issue or a post on a discussion thread.

Enhancements to the new mark mechanism

In June 2022 we introduced a new mark mechanism [2, p. 76] that allows keeping track of multiple independent marks. It also properly supports top marks, something that wasn't reliably possible with LaTeX before.

There was, however, one limitation: to retrieve the marks from the page data it was necessary to `\vsplit` that data artificially so that TeX would produce split marks that the mechanism could then use. Unfortunately, TeX gets very upset if it finds infinite negative glue (e.g., from `\vss`) within this data. This is not totally surprising because such glue would allow splitting off any amount of material as such glue would hide its size. TeX therefore responds with an error message if it find such glue while doing a `\vsplit` operation (and it does so even if a later glue item cancels the infinite glue).

To account for this, the code in 2022 attempted to detect this situation beforehand and if so did not do any splitting but, of course, it would then also not extract any mark information.

In this release the approach has been changed and we always do a `\vsplit` operation and thus always get the right mark data extracted. While it is not possible to avoid upsetting TeX in case we have infinite negative glue present, it is possible to hide this (more or less) from the user.[2] With the new code TeX will neither stop nor show anything on the terminal. What we can't do, though, is avoid an error being written to the log file, but to make it clear that this error is harmless and should be ignored we have arranged the code so that the error message, if it is issued, takes the following format:

```
! Infinite glue shrinkage found in box being split.
<argument> Infinite shrink error above ignored !
l. ... }
```

Not perfect (especially the somewhat unmotivated `<argument>`), but you can only do so much when error messages and their texts are hard-wired in the engine.

So why all this? There are two reasons: we do not lose marks in edge cases any more, and perhaps more importantly we are now also reliably able to extract marks from arbitrarily boxed data, something that wasn't possible at all before. This is necessary, for example, to support extended marks in `multicols` environments or extract them from floats, marginpars, etc.

Details about the implementation can be found in `texdoc ltmarks-code` or in the shorter `texdoc ltmarks-doc` (which only describes the general concepts and the command interfaces).

Providing xtemplate in the format

In LaTeX News 32, we described the move of one long-term experimental idea into the kernel: the package xparse, which was integrated as ltcmd. With this edition, we move another long-term development idea to stable status: *templates*.

In this context, templates are a mechanism to abstract out various elements of a document (such as "sectioning") in such a way that different implementations can be interchanged, and design decisions can be implemented efficiently and controllably.

In contrast to ltcmd, which provides a mechanism that many document authors will exploit routinely, templates are a more specialised tool. We anticipate that they will be used by a small number of programmers, providing generic ideas that will then be used within document classes. Most document authors will therefore likely directly encounter templates only rarely. We anticipate

[2] A note to l3build users that make use of its testing capabilities: the new mechanism temporarily changes `\interactionmode` and, for implementation reasons in TeX, that results in extra newlines in the `.log` file, so instead of seeing `[1] [2]` you will see each on separate lines. This means that test files might show differences of that nature, once the code is active, and must therefore be regenerated as necessary.

though that they will be *using* templates provided by the team or others.

The template system requires three separate ideas

- Template *type*: the "thing" we are using templates for, such as "sectioning" or "enumerated-list"
- A template: a combination of code and keys that can be used to implement a type. Here for example we might have "standard-LaTeX-sectioning" as a template for "sectioning"
- One or more *instances*: a specific use case of a template where (some) keys are set to known values. We might for example see "LaTeX-section", "LaTeX-subsection", etc.

As part of the move from the experimental xtemplate to kernel integration, the team have revisited the commands provided. The stable set now comprises

- `\NewTemplateType`
- `\DeclareTemplateInterface`
- `\DeclareTemplateCode`
- `\DeclareTemplateCopy`
- `\EditTemplateDefault`
- `\UseTemplate`
- `\DeclareInstance`
- `\DeclareInstanceCopy`
- `\EditInstance`
- `\UseInstance`
- `\IfInstanceExistsTF` and variants

To support existing package authors, we have released an updated version of xtemplate which will work smoothly with the new kernel-level code. The existing commands provided in xtemplate will continue to work, but we encourage programmers to move to the set above.

New or improved commands

doc: Provide `\ProvideDocElement`

In addition to `\NewDocElement` and `\RenewDocElement` we now also offer a `\ProvideDocElement` declaration that does nothing unless the doc element could be declared with `\NewDocElement`. This can be useful if documentation files are processed both individually and combined.

doc: Better support for upquote

In LaTeX News 37 [6] we wrote that support for the upquote package was added to the doc package, but back then this was added only for `\verb` and the `verbatim` environments. However, in a typical `.dtx` file, most of the code will be in the body of some `macrocode` or `macrocode*` environments, and neither of these was affected by adding upquote. We have now updated doc so that upquote alters the quote characters in these environments as well. *(github issue 1230)*

ifthen: Guard against active characters in comparisons

The `\ifthenelse` command now ensures that `<`, `=` and `>` are safe in numeric tests, even if they have been made active (typically by babel language shorthands). *(github issue 756)*

New conditionals: `\IfClassAtLeastT` and friends

Around 2020 we added a number of conditionals with CamelCase names, i.e., `\IfClassAtLeastTF`, `\IfClassLoadedTF`, `\IfClassLoadedWithOptionsTF`, `\IfFormatAtLeastTF`, `\IfPackageAtLeastTF`, `\IfPackageLoadedTF`, and `\IfPackageLoadedWithOptionsTF` to help arranging conditional code that depends on the release of a particular class, package or format. However, we only provided the `TF` commands and not also the `T` and `F` variants. This has now been changed.

In 2023 we introduced `\IfFileAtLeastTF` but we did not also provide `\IfFileLoadedTF` at the same time. This conditional and its `T` and `F` variants have now also been added. Remember that one can only test for files that contain a `\ProvidesFile` line. We did the same for the conditionals `\IfLabelExistsTF` and `\IfPropertyExistsTF`, also introduced in 2023.[3] *(github issues 1222 and 1262)*

Code improvements

Load packages only at the top level

Classes and packages must be loaded only by using the commands `\documentclass` and `\usepackage` or the class interface commands such as `\LoadClass` or `\RequirePackageWithOptions`; moreover, all of these must always be used at the top level, and not inside a group of any type (for example, within an environment). Previously LaTeX did not check this, which would often lead to low level errors later on if package declarations were reverted when a group ended. LaTeX now checks the group level and an error is thrown if the class or package is loaded in a group. *(github issue 1185)*

Keep track of lost glyphs

A while ago we changed the LaTeX default value for `\tracinglostchars` from `1` to `2` so that missing glyphs generate at least a warning, but we forgot to make the same change to `\tracingnone`. Thus, when issuing that command LaTeX stopped generating warnings

[3] By mistake they were initially introduced under the names `\IfLabelExistTF` and `\IfPropertyExistTF`; we corrected that at the same time. This is a breaking change, but the commands have been used so far only in kernel code.

about missing glyphs. This has now been corrected. *(github issue 549)*

Improve fontenc error message

If the fontenc package is asked to load a font encoding for which it doesn't find a suitable `.def` file then it generates an error message indicating that the encoding name might be misspelled. That is, of course, one of the possible causes, but another one is that the installation is missing a necessary support package, e.g., that no support for Cyrillic fonts has been installed. The error message text has therefore been extended to explain the issue more generally. *(github issue 1102)*

Warn if counter names are problematic

In the past it was possible to declare, for example, `\newcounter{index}` with the side-effect that this defines `\theindex`, even though LaTeX has a `theindex` environment that then got clobbered by the declaration. This has now been changed: if `\the`⟨*counter*⟩ is already defined it is not altered, but instead a warning message is displayed. *(github issue 823)*

Extended information in `\listfiles`

The `\listfiles` command provides useful information when finding issues related to variation in package versions. However, this has to date relied on the information in the `\ProvidesPackage` line, or similar: that can be misleading if for example a file has been edited locally. We have now extended `\listfiles` to take an optional argument which can include the MD5 hash and size of each file in the `.log`. Thus for example you can use

```
\listfiles[hashes,sizes]
```

to get both the file sizes and file hashes in the `.log` as well as the standard release information. *(github issue 945)*

Optimize creation of simple document commands

Creating document commands using declarations such as `\NewDocumentCommand`, etc., provides a very flexible way of grabbing arguments. When the document command only takes simple mandatory arguments, this has to-date added an overhead that could be avoided. We have now refined the internal code path such that "simple" document commands avoid almost any overhead at point-of-use, making the results essentially as efficient as using `\newcommand` for low-level TeX constructs. Note that as `\NewDocumentCommand` makes engine-robust commands, the direct equivalent to `\newcommand` is `\NewExpandableDocumentCommand`. *(github issue 1189)*

Handling of end-of-lines in +v arguments of `\NewDocumentCommand` and friends

The `+v` argument type provided by declarations such as `\NewDocumentCommand`, etc., allows grabbing of multiple lines of text in a verbatim-like argument. Almost always, the result of this grabbing will be used in a typesetting context. Previously, the end-of-line characters were stored literally as category code 12 ("other") `^^M` tokens. However, these are difficult to work with in general. We have now revised this behavior, such that end-of-line characters are converted to the `\obeyedline` command when parsed by `+v`-type arguments. This change may require adjustments to the source of some documents, but the enhanced ability of users and programmers to exploit the `+v`-type argument means we believe it is necessary.

Declaring appropriate sub-encodings for TS1 symbol fonts

In 2020 we incorporated support for the `TS1` symbol encoding directly into the kernel and in this way removed the need to load the textcomp package [3] to make commands such as `\texteuro` available.

There is, however, a big problem with this `TS1` symbol encoding: only very few fonts provide every glyph that is supposed to be part of `TS1`. This means that changing font families might result in certain symbols becoming unavailable. This can be a major disaster if, for example, the symbol `\texteuro` (€) or `\textohm` (Ω) no longer gets printed in your document, just because you altered the text font family.

To mitigate this problem, in 2020 we also introduced the declaration `\DeclareEncodingSubset`. This declaration is supposed to be used in font definition files for the `TS1` encoding to specify which subset (we have defined 10 common ones) a specific font implements. If such a declaration is used then missing symbols are automatically taken from a fallback font.

While this is not perfect, it is the best you can do other than painstakingly checking that your document uses only glyphs that the font supports and, if necessary, switching to a different font or avoiding the missing symbols. See also the discussion in [4].

To jumpstart the process we also added declarations to the LaTeX kernel for most of the fonts found in TeX Live at the time—with the assumption that such declarations would over time be superseded by declarations in the `.fd` files. Unfortunately, this hasn't happened yet (or not often) and so many of the initial declarations went stale: several fonts got new glyphs added to them (so their sub-encoding should have been changed but didn't); others (mainly due to license issues) changed the family name and thus our declarations became useless and the renamed fonts (now without a declaration) ended up in the default sub-encoding that offers only a few glyphs; yet others such as CharisSIL (which triggered the GitHub issue) were simply not around at the time.

We have, therefore, again attempted to provide the (currently) correct declarations, but it is obvious that this is not a workable process. As we do not maintain the

fonts we do not have the information that something has changed, and to regularly check the ever growing font support bundles is simply not possible. It is therefore very important that maintainers of font packages not only provide `.fd` files but also add such a declaration to every `TS1...fd` font definition file that they distribute.

To simplify this process, we now provide a simple LaTeX file (`checkencodingsubset.tex`) for determining the correct (safe) sub-encoding. If run, it asks for a font family and then outputs its findings, for example, for `AlgolRevived-TLF` you will get:

```
----------------------------------------
Testing font family AlgolRevived-TLF
(currently TS1-sub-encoding 9)
----------------------------------------
Some glyphs are missing from sub-encoding 8:
   ==> \textcelsius (137) is missing
   ==> \texttwosuperior (178) is missing
   ==> \textthreesuperior (179) is missing
   ==> \textonesuperior (185) is missing
Some glyphs are missing from sub-encoding 7:
   ==> \texteuro (191) is missing
All glyphs between sub-encoding 6 and 7 exist
All glyphs between sub-encoding 5 and 6 exist
All glyphs between sub-encoding 4 and 5 exist
Some glyphs are missing from sub-encoding 3:
   ==> \textwon (142) is missing
All glyphs between sub-encoding 2 and 3 exist
Some glyphs are missing from sub-encoding 1:
   ==> \textmho (77) is missing
   ==> \textpertenthousand (152) is missing
All glyphs between sub-encoding 0 and 1 exist
All glyphs in core exist
----------------------------------------
TS1 encoding subset for AlgolRevived-TLF (ok)
Use sub-encoding 9
----------------------------------------
```

This output is meant for human consumption, e.g., you see which glyphs are missing and why a certain sub-encoding is suggested, but it is not that hard to use it in a script and extract the suggested sub-encoding by grepping for the line starting with `Use sub-encoding`.

Of course, this check will only work if the missing glyphs are really missing: some fonts placed "tofu"[4] into such slots and in this case it looks to TeX as if the glyph is provided. For example, for the old Palatino fonts (family `ppl`) it would report

```
----------------------------------------
TS1 encoding subset for ppl (bad)
Use sub-encoding 0 (not 5)
----------------------------------------
```

[4] Little squares to indicate a missing symbol.

thus it claims that all glyphs are provided, while in reality more than twenty are missing and sub-encoding 5, as declared in the kernel, is in fact correct. *(github issue 1257)*

Behavior when loading textcomp without options

When incorporating the textcomp package into the LaTeX kernel, in the February 2020 release [3], the default type of its package messages was changed from package info (`Package textcomp Info`) to LaTeX kernel info (`LaTeX Info`). But if textcomp was loaded without options, the message type got restored to package info. This restoration has now been canceled.

Note that loading textcomp with one of the options `error`, `warn`, or `info` still changes the message type to an error, warning, or info message from the textcomp package. *(github issue 1333)*

Rollback improvements

When requesting a rollback of the LaTeX kernel and/or packages, several packages produced the error "Suspicious rollback date" because their rollback section contained only data about recent releases even if the package, such as array, was available since the first release of LaTeX 2$_\varepsilon$ in 1994. We now suppress this error and load the first release that is still part of the distribution (and hope for the best). This change was implemented for the packages amsmath, array, doc, graphics, longtable, multicol, showkeys, textcomp, and varioref. *(github issue 1333)*

Documentation improvements

Further updates to the guides

We reported about the updated versions of `usrguide` and `clsguide` in LaTeX News 37 [6]. We have now revised `fntguide` as well to reflect the changes and macros added to the kernel over the last years of development. Note that the file name hasn't changed and there is no `fntguide-historic`.

Bug fixes

Fix inconsistent expansion of the package option list

LaTeX applies one-step expansion to the raw option list of packages and classes, so that constructions such as

```
\def\myoptions{opt1,opt2}
\usepackage[\myoptions]{foo}
```

are supported. But if a package declares its options using the new key/value approach [5] and it gets loaded a second time, then its raw option list will not be expanded and so an error might be raised. This has now been corrected. *(github issue 1298)*

Fix logic for first mark (page region)
In the new mark mechanism introduced in June 2022 [5] the result of `\FirstMark` on a two-column page was incorrect if the first column contained no marks. In that case it should have returned the first mark of the second column but didn't. This has now been corrected.

Documents using `\leftmark` are not affected, because that command is still using the old mechanism for now. *(github issue 1359)*

Struts at the end of footnotes or p columns
To produce consistent spacing in footnotes and tabular p-cells LaTeX adds a strut at the beginning and end of the content. This assumed, however, that the content of the footnote or tabular cell ended in horizontal mode and so, until now, these struts were unconditionally added; as a result, if this content ended with vertical material then this strut started a new paragraph consisting of a single line with just the strut in it. This has finally been corrected and now the placement logic for the strut changes when vertical mode is detected.

(First seen in a bug report for footmisc in combination with bigfoot)

Fix a "missing `\item`" rollback error
If LaTeX is rolled back to a date between 2023/06/01 (inclusive) and 2024/06/01 (exclusive), any list-based environment would raise an error:

```
! LaTeX Error: Something's wrong--perhaps
a missing \item.
```

This has now been corrected as a hotfix in patch level 2, by enhancing the 2023/06/01 version rollback code of the new paragraph mechanism. *(github issue 1386)*

Changes to packages in the amsmath category

amsmath: Correct equation tag placement
If there is not enough space to place an equation tag on the same line as the equation then amsmath calculates a suitable offset and it places the tag above (or below) the equation. In the case of the `gather` environment this offset was not reset at the end, with the result that it also got applied to any following environment, resulting in incorrect spacing in certain situations. This has now been corrected. *(github issue 1289)*

Changes to packages in the tools category

array, longtable, tabularx: Support tagging
These three packages have been extended so they can now, on request, produce tagged tabular. This is done by adding a number of sockets (see [7]) that, by default, do nothing; but when tagged PDF is requested they get equipped with appropriate plugs.

In the previous LaTeX release this was handled in `latex-lab`, by patching the packages when tagging was requested.

array: No `\unskip` in math cells
Math cells in the standard `array` environment of the kernel are not subject to space removal at the right end of the cell, i.e., explicit spaces from `\hspace` or `\␣`, etc. are honored (normal spaces are automatically ignored in math). In the array package all spaces got removed by calling `\unskip` unconditionally, regardless of the type of cell. This difference in behavior has now been removed by correcting the processing of math cells in array. *(github issue 1323)*

verbatim: `\verb` showed visible spaces
A recent change in the kernel was not reflected in the verbatim package, with the result that `\verb` showed visible spaces (␣) after the package was loaded. This has already been corrected in a hotfix for the November 2023 release. *(github issue 1160)*

verbatim: Support tabs in `\verbatiminput*`
Mimicking the November 2023 kernel update that allowed `\verb*` to mark tabs as spaces, the verbatim package has now been updated so that `\verbatiminput*` also marks tabs as spaces. *(github issue 1245)*

multicol: `\columnbreak` interferes with mark mechanism
The multicol package has to keep track of marks (from `\markright` or `\markboth`) as part of its output routine code and can't rely on LaTeX handling that automatically. It does so by artificially splitting page data with `\vsplit` to extract the mark data. With the introduction of `\columnbreak` that code failed sometimes, because it was not seeing any mark that followed such a forced column break.

This has now been corrected, but there is further work to do, because as of now multicol does not yet handle marks using the new mark mechanism—see the discussion at the beginning of the newsletter. *(github issue 1130)*

showkeys: Allow `\newline` in amsthm to work
Previously showkeys added an extra box layer which disabled the `\newline` of amsthm theorem styles. This extra box has now been avoided. *(github issue 1123)*

xr: Support links and properties
The xr package implements a system for eXternal References. The xr-hyper package (in the hyperref bundle) extended this to also support links to external documents. Using last year's extension of the `\label` command, which unified the label syntax of LaTeX and hyperref, it became possible to merge the two packages and thus make xr-hyper obsolete. With this

change it is also possible to refer to properties that are stored in external documents using `\RecordProperties`. *(github issue 1180)*

Changes to files in the cyrillic category

Correct definition of `\k`

Ages ago, the encoding-specific definitions for various accent commands were changed to guard against altering some parameter values non-locally by mistake. For some reason the definition for `\k` in the Cyrillic encodings `T2A`, `T2B`, and `T2C` didn't get this treatment. This oversight has now been corrected. *(github issue 1148)*

References

[1] Leslie Lamport. *LaTeX: A Document Preparation System: User's Guide and Reference Manual*. Addison-Wesley, Reading, MA, USA, 2nd edition, 1994. ISBN 0-201-52983-1. Reprinted with corrections in 1996.

[2] LaTeX Project Team. *LaTeX 2ε news 1–39*. June, 2024. https://latex-project.org/news/latex2e-news/ltnews.pdf

[3] LaTeX Project Team. *LaTeX 2ε news 31*. February, 2020. https://latex-project.org/news/latex2e-news/ltnews31.pdf

[4] LaTeX Project Team. *LaTeX 2ε news 33*. June 2021. https://latex-project.org/news/latex2e-news/ltnews33.pdf

[5] LaTeX Project Team. *LaTeX 2ε news 35*. June 2022. https://latex-project.org/news/latex2e-news/ltnews35.pdf

[6] LaTeX Project Team. *LaTeX 2ε news 37*. June 2023. https://latex-project.org/news/latex2e-news/ltnews37.pdf

[7] LaTeX Project Team. *LaTeX 2ε news 38*. November 2023. https://latex-project.org/news/latex2e-news/ltnews38.pdf

[8] Frank Mittelbach and Ulrike Fischer. *Enhancing LaTeX to automatically produce tagged and accessible PDF*. *TUGboat* 45:1, 2024. https://latex-project.org/publications/indexbyyear/2024/